ENCONTRO 2 · REDES NEURAIS NO KERAS

Rede densa e rede convolucional

Usando o Keras com o conjunto de imagens MNIST

VISÃO GERAL

O que vamos fazer hoje

- Construir duas redes neurais no Keras: uma rede densa e uma rede convolucional
- Usar o mesmo conjunto de imagens (MNIST) nas duas, para poder compará-las
- Testar as duas com dígitos escritos à mão pelos próprios alunos

Foco: usar o Keras. A teoria entra apenas no momento em que é necessária.

VISÃO GERAL

A receita do Keras

Todo modelo no Keras segue os mesmos 7 passos. Nas duas redes de hoje, só o passo 3 muda.

1	2	3	4	5	6	7
Carregar os dados	Preparar os dados	Montar o modelo	Compilar	Treinar	Avaliar	Prever

O passo 3 é o único que muda entre a rede densa e a rede convolucional.

TERMOS BÁSICOS

O que é o Keras

Keras

- Uma caixa de peças prontas para montar redes neurais em Python
- Em vez de escrever as contas, você descreve a rede: “uma camada com 128 neurônios”

TensorFlow

- O motor por baixo, que faz as contas pesadas
- Já vem instalado no Google Colab

O Keras descreve a rede; o TensorFlow a executa.

TERMOS BÁSICOS

O que é o MNIST

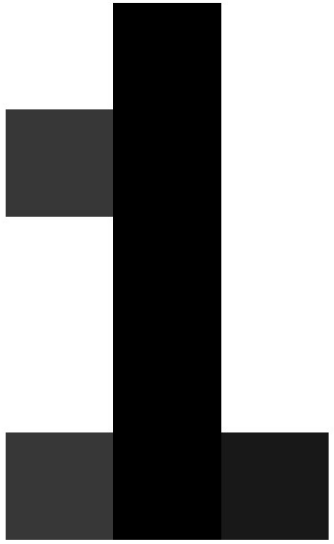
- Um conjunto de dados, não um programa
- 70 mil imagens de números escritos à mão, de 0 a 9
- Cada imagem vem com a resposta certa (o rótulo)
- 60 mil imagens para treino e 10 mil para teste
- O “primeiro exercício” clássico das redes neurais

O arquivo não está dentro do Keras: mnist.load_data() o baixa da internet na primeira vez.

TERMOS BÁSICOS

Pixel: a imagem é uma tabela de números

O que vemos



O que o computador recebe

0	0	255	0	0
0	200	255	0	0
0	0	255	0	0
0	0	255	0	0
0	200	255	230	0

- Cada pontinho da imagem é um pixel
- 0 = sem tinta; 255 = traço mais forte
- Uma imagem do MNIST tem 28 × 28 = 784 pixels

Uma imagem é uma tabela de números.

TERMOS BÁSICOS

Treino e teste

Treino

60 mil imagens com a resposta, para a rede aprender

`x_treino, y_treino`

Teste

10 mil imagens guardadas à parte, que a rede nunca viu

`x_teste, y_teste`

O teste é como uma prova com questões novas: mede se a rede aprendeu de verdade.

Passo 1 · Carregar os dados

```
from tensorflow.keras.datasets import mnist

(x_treino, y_treino), (x_teste, y_teste) = mnist.load_data()

print(x_treino.shape, x_teste.shape)
```

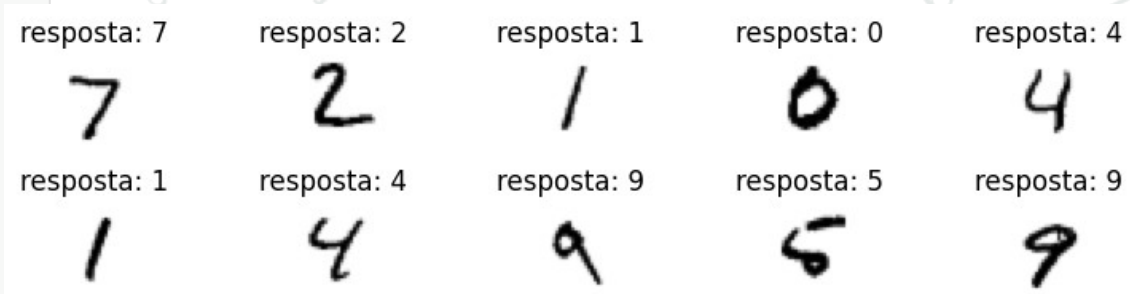
Saída esperada: (60000, 28, 28) (10000, 28, 28), isto é, 60 mil imagens de 28 × 28.

Variável	O que guarda	Formato
x_treino	as imagens de treino	(60000, 28, 28)
y_treino	a resposta certa de cada uma	(60000,)
x_teste	as imagens de teste	(10000, 28, 28)
y_teste	a resposta certa de cada uma	(10000,)

Passo 1 · Ver as imagens

```python
import matplotlib.pyplot as plt

fig, eixos = plt.subplots(2, 5, figsize=(10, 4.5))
for i, eixo in enumerate(eixos.ravel()):
    eixo.imshow(x_treino[i], cmap="gray_r")
    eixo.set_title(f"resposta: {y_treino[i]}")
    eixo.axis("off")
plt.tight_layout()
plt.show()
```



Passo 2 · Preparar os dados

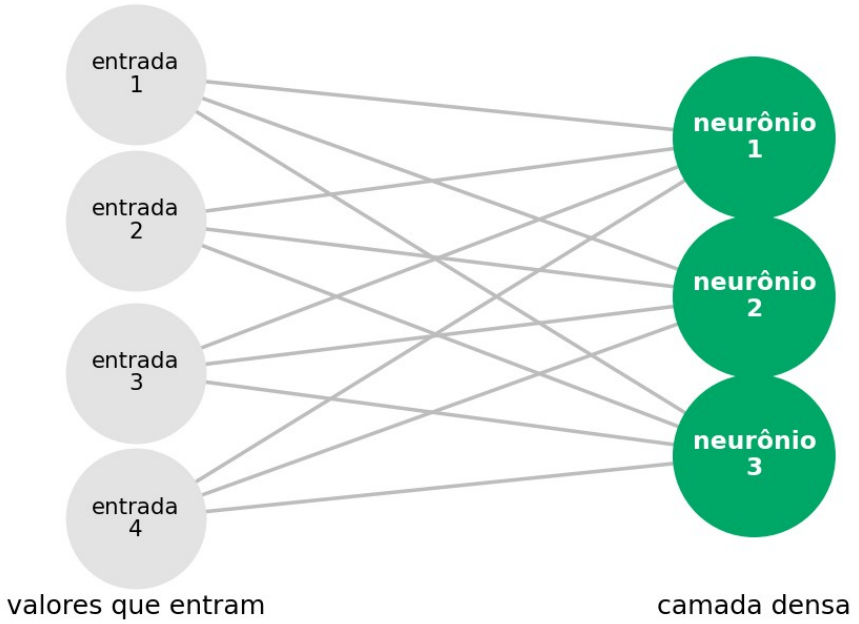
```
x_treino_n = x_treino / 255.0
x_teste_n = x_teste / 255.0

print(x_treino_n.min(), x_treino_n.max())
```

Saída esperada: 0.0 1.0 (o menor e o maior valor depois da divisão)

- Os pixels vão de 0 a 255; a rede aprende melhor com valores de 0 a 1
- Dividir por 255.0 coloca todos os valores nessa faixa
- É a mesma ideia da normalização usada no KNN e nas redes do Encontro 1
- A foto de teste, mais adiante, precisará passar pela mesma divisão

O que é uma rede densa



cada neurônio olha todas as entradas

- Cada neurônio recebe todos os valores da camada anterior
- É o mesmo neurônio do Encontro 1 (reta + dobra), agora em maior número
- No Keras: **`layers.Dense(...)`**

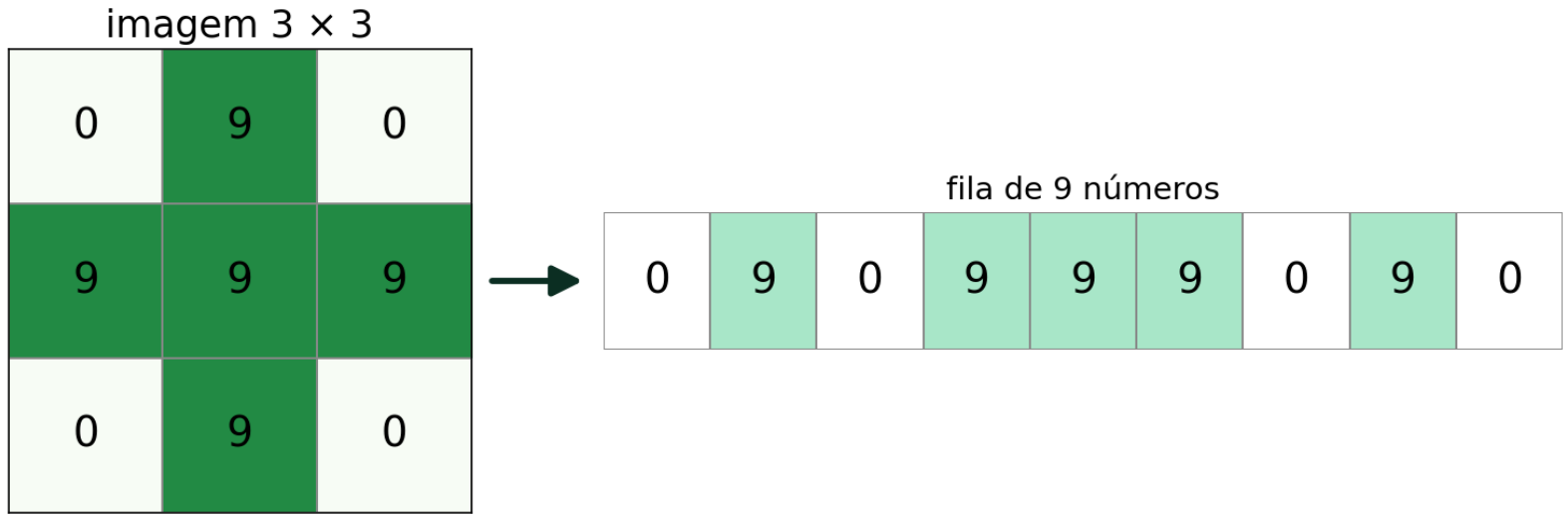
Passo 3 · Montar o modelo (rede densa)

```
from tensorflow import keras
from tensorflow.keras import layers

modelo_densa = keras.Sequential([
    keras.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
```

Camada	O que faz
Sequential	pilha de camadas
Input(28, 28)	a imagem que entra
Flatten	estica a imagem em fila
Dense(128, relu)	128 neurônios
Dense(10, softmax)	10 saídas, uma por dígito

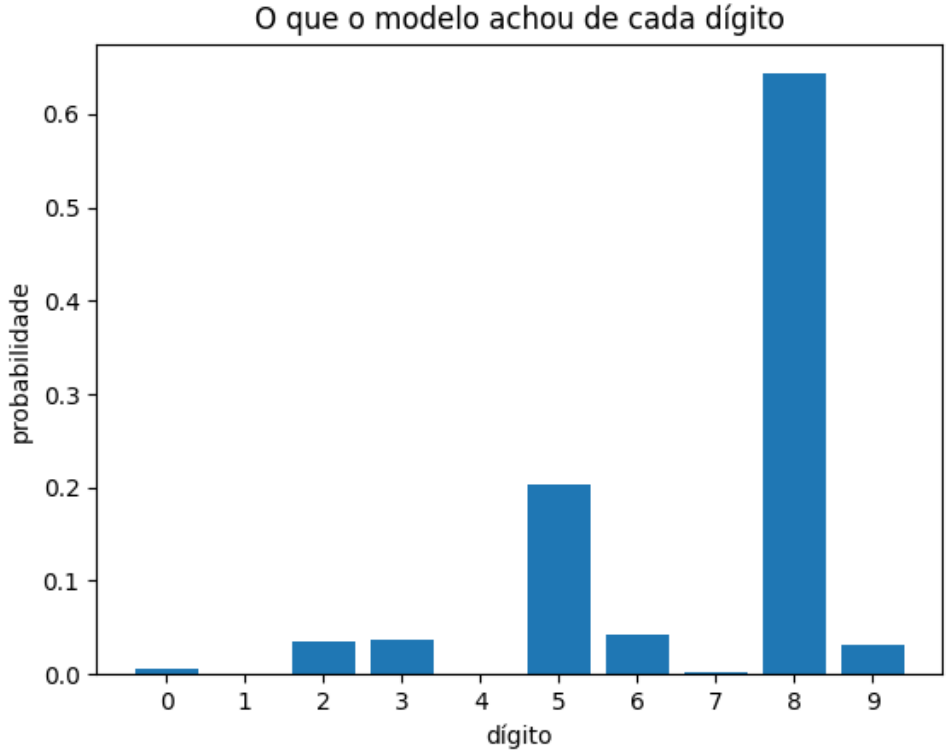
Flatten: a imagem vira uma fila de números



Na aula: 28 × 28 imagem → fila de 784 números

A camada densa espera uma fila de números; o Flatten estica a imagem de 28 × 28 em 784 valores.

Softmax: dez saídas, uma probabilidade por dígito



Exemplo real: as 10 saídas da rede para a foto de um 6 escrito à mão. A rede respondeu 8, com 64%.

- A última camada tem 10 neurônios, um por dígito (0 a 9)
- A função softmax transforma as 10 saídas em probabilidades que somam 1
- A rede responde com o dígito de maior probabilidade

Passo 4 · Compilar

```
modelo_densa.compile(optimizer="adam",
                     loss="sparse_categorical_crossentropy",
                     metrics=["accuracy"])
```

Parâmetro	O que define
optimizer="adam"	o método que ajusta os pesos da rede
loss="sparse_categorical_crossentropy"	a medida do erro que a rede tenta reduzir
metrics=["accuracy"]	o que queremos acompanhar: a acurácia

Época e acurácia

Uma época é uma passada completa por todas as 60 mil imagens de treino.



- A cada época, a rede ajusta seus pesos e a acurácia costuma subir
- **Acurácia** = porcentagem de acertos

epochs=5 significa 5 passadas pelos dados de treino.

Passo 5 · Treinar

```
modelo_densa.fit(x_treino_n, y_treino, epochs=5)
```

fit recebe as imagens, as respostas e o número de épocas.

Passo 6 · Avaliar

```
perda, acuracia = modelo_densa.evaluate(x_teste_n, y_teste, verbose=0)
print(f"Acurácia no teste: {acuracia:.2%}")
```

97,29%

acurácia no teste, obtida no Colab (pode variar um pouco a cada execução)

- evaluate mede o desempenho nas 10 mil imagens de teste
- São imagens que a rede nunca viu durante o treino
- Esse número é o que importa, não a acurácia do treino

Passo 7 · Prever

```python
import numpy as np
import matplotlib.pyplot as plt


previsoes = np.argmax(modelo_densa.predict(x_teste_n, verbose=0), axis=1)


fig, eixos = plt.subplots(2, 5, figsize=(10, 4.5))
for i, eixo in enumerate(eixos.ravel()):
    eixo.imshow(x_teste[i], cmap="gray_r")
    cor = "green" if previsoes[i] == y_teste[i] else "red"
    eixo.set_title(f"previsto: {previsoes[i]} | certo: {y_teste[i]}", color=cor)
    eixo.axis("off")
plt.tight_layout()
plt.show()
```

predict dá as 10 probabilidades; argmax escolhe a maior; essa é a resposta da rede.

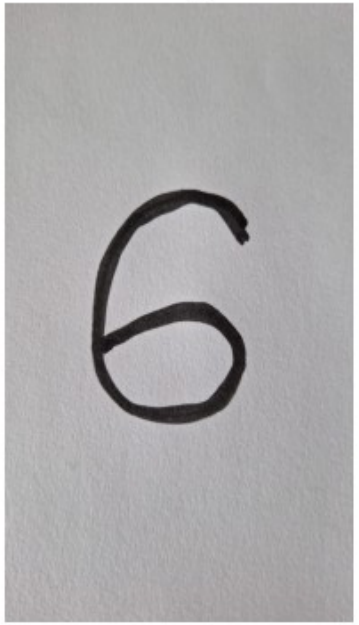
Por que a foto precisa ser convertida

	MNIST	Foto de papel
Traço	claro sobre fundo escuro	escuro sobre fundo claro
Tamanho	28 × 28 pixels	milhares de pixels, colorida
Posição	dígito centralizado, com margem	posição e tamanho livres
Fundo	totalmente limpo (valor 0)	papel acinzentado, sombras e ruído

A foto precisa ser convertida para o mesmo formato com que a rede foi treinada.

O caminho da foto até 28 × 28

foto original



28 x 28 (formato do MNIST)



Exemplo: foto de um 6 e a versão 28 × 28 (primeira versão da conversão, antes do passo 7 ao lado).

1. Tons de cinza e mais contraste
2. Inverter: traço claro sobre fundo escuro
3. Limpar o fundo
4. Recortar no traço
5. Reduzir para caber em 20 × 20
6. Colocar numa tela 28 × 28
7. Centralizar pelo centro de massa

Enviar a foto do celular ao Colab

1. Escrever o dígito com caneta grossa, em papel claro e com boa luz
2. Fotografar e, se preciso, recortar a foto para ficar só o dígito
3. Enviar a foto para si mesmo (WhatsApp Web, e-mail ou Google Fotos)
4. Baixar a foto no computador
5. Executar a célula abaixo e escolher o arquivo

```
import io
from PIL import Image
from google.colab import files

enviado = files.upload()          # abre a janela para escolher a foto
nome = list(enviado.keys())[0]
original = Image.open(io.BytesIO(enviado[nome]))
```

O código da conversão

```python
import numpy as np
from PIL import Image, ImageOps
from scipy import ndimage


def preparar_imagem(img):
    img = ImageOps.exif_transpose(img).convert("L")   # corrige a rotação e passa para cinza
    img = ImageOps.autocontrast(img)                  # reforça o contraste
    a = 255 - np.array(img)                           # inverte: traço claro, fundo escuro
    a = np.where(a > 100, a, 0)                       # limpa o fundo
    ys, xs = np.where(a > 0)
    a = a[ys.min():ys.max() + 1, xs.min():xs.max() + 1]   # recorta no traço
    h, w = a.shape
    k = 20 / max(h, w)                                # o dígito cabe em 20 x 20
    novo = Image.fromarray(a.astype(np.uint8)).resize((max(1, round(w * k)), max(1, round(h * k))))
    tela = Image.new("L", (28, 28), 0)                # tela 28 x 28, fundo preto
    tela.paste(novo, ((28 - novo.width) // 2, (28 - novo.height) // 2))
    a = np.array(tela)
    cy, cx = ndimage.center_of_mass(a)                # onde está o "peso" da tinta
    return ndimage.shift(a, (14 - cy, 14 - cx), order=1)   # centraliza por esse ponto
```

Cada linha tem um comentário. O notebook já traz a função pronta.

Testar a foto no modelo

```
import matplotlib.pyplot as plt

imagem_28 = preparar_imagem(original)

plt.imshow(imagem_28, cmap="gray_r")
plt.axis("off")
plt.show()
```

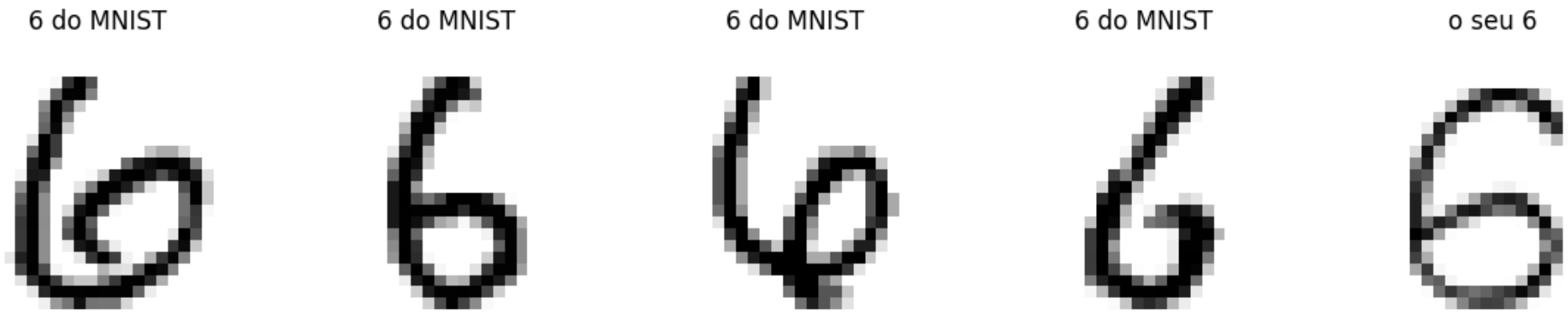
- preparar_imagem converte a foto
- O gráfico mostra o que a rede vai receber

```
import numpy as np

entrada = (imagem_28 / 255.0).reshape(1, 28, 28)
probabilidades = modelo_densa.predict(entrada, verbose=0)
digito = int(np.argmax(probabilidades))

print(f"O modelo acha que é: {digito} ({probabilidades[0][digito]:.0%} de confiança)")
```

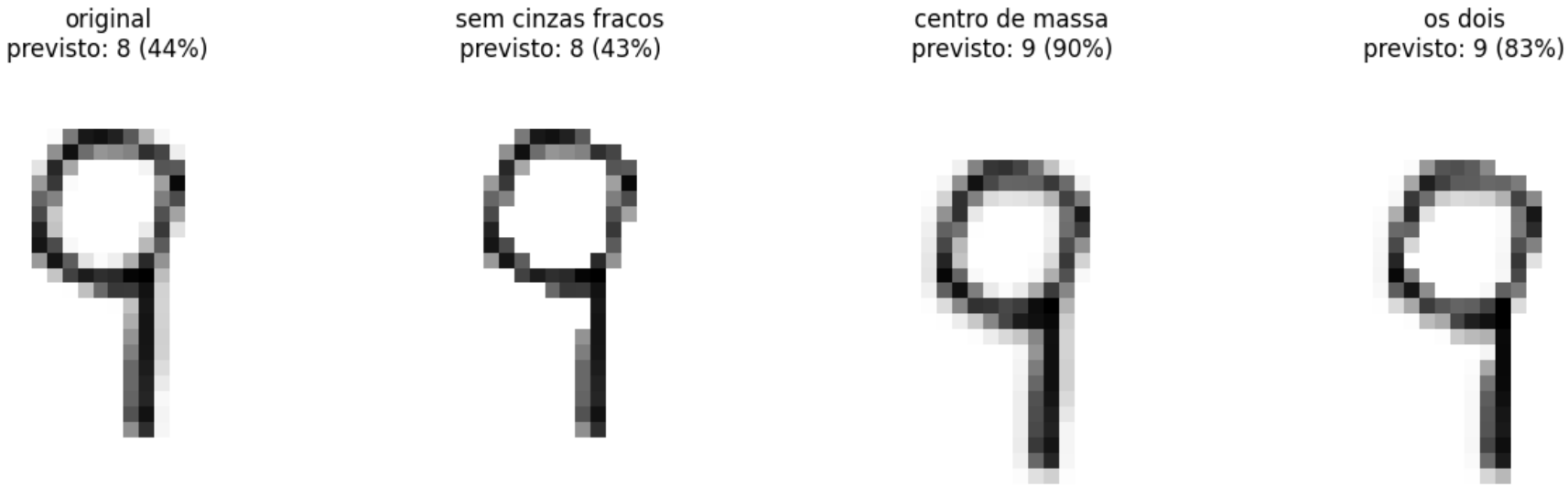
Onde a rede densa erra: o 6



O último é o 6 escrito à mão pelo professor.

- O 6 do MNIST tem traço mais grosso; o da foto, mais fino
- O topo da foto é um arco que curva para baixo, em vez de uma haste
- Previsão com a primeira conversão: 8 (64%); o 6 ficou em 3º lugar, com 4%
- Engrossar o traço não ajudou: continuou 8 (75%)

Onde a rede densa erra: o 9 e a posição



original: 8 (44%) · sem cinzas fracos: 8 (43%) · centro de massa: 9 (90%) · os dois: 9 (83%)

Deslocar o dígito poucos pixels mudou a resposta: a rede densa trata cada pixel de forma separada.

Resumo do Bloco 1

- A receita do Keras tem 7 passos
- Uma imagem é uma tabela de números; a rede densa a recebe esticada em uma fila de 784
- Com 128 neurônios, a rede acertou cerca de 97% das imagens de teste do MNIST
- Com fotos escritas à mão, ela errou alguns dígitos, e a posição do dígito mudou a resposta
- A rede densa trata cada pixel de forma separada

Próximo bloco: uma rede que olha a imagem por pedaços, a rede convolucional.

Por que uma rede convolucional

- A rede densa estica a imagem em uma fila e perde a vizinhança dos pixels
- Um pixel e o vizinho ao lado passam a ser apenas dois números sem relação
- Por isso o mesmo 9, deslocado poucos pixels, mudou a resposta

A rede convolucional olha a imagem por pedaços pequenos, preservando a vizinhança.

Convolução: um filtro que desliza pela imagem

imagem 6 × 6 (uma barra vertical)

0	0	9	9	0	0
0	0	9	9	0	0
0	0	9	9	0	0
0	0	9	9	0	0
0	0	9	9	0	0
0	0	9	9	0	0

filtro 3 × 3

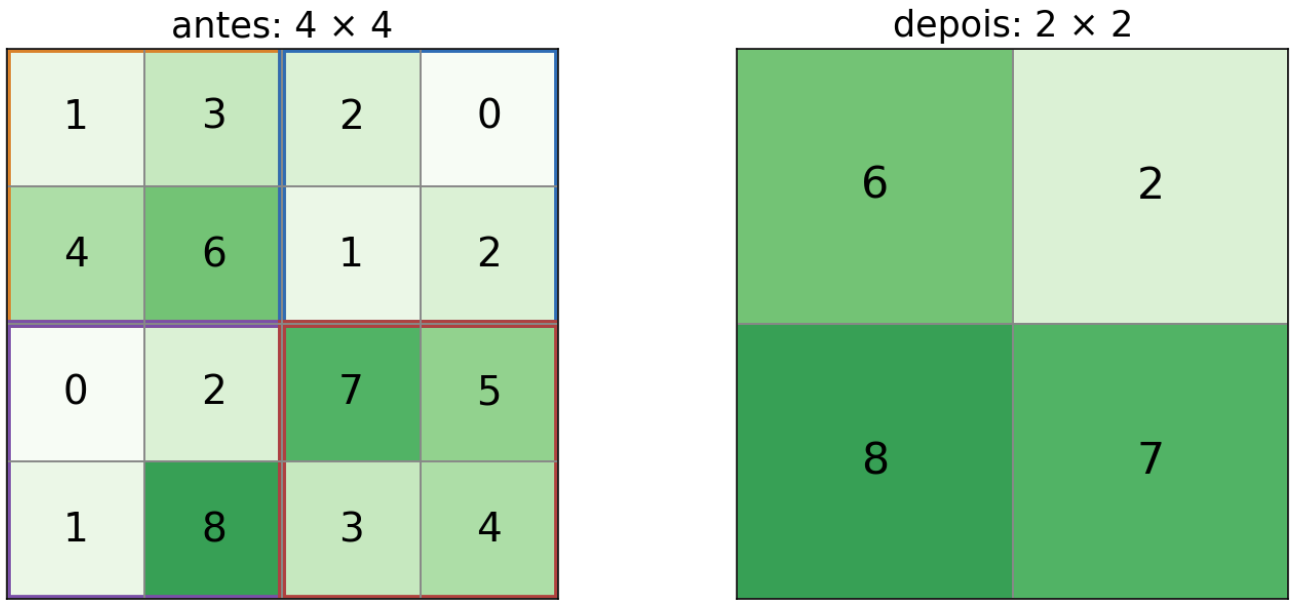
-1	0	1
-1	0	1
-1	0	1

resultado 4 × 4

27	27	-27	-27
27	27	-27	-27
27	27	-27	-27
27	27	-27	-27

O filtro desliza pela imagem e mede quanto cada pedaço se parece com um padrão (aqui, uma borda vertical). Na rede, os filtros são aprendidos no treino.

Pooling: reduzir mantendo o sinal mais forte



cada bloco 2 × 2 fica só com o seu maior valor

Com pooling 2 × 2, a imagem fica com metade da altura e da largura, guardando o maior valor de cada bloco.

Canal e formato da entrada

Imagem	Canais	Formato no Keras
Em tons de cinza (MNIST)	1	(28, 28, 1)
Colorida	3 (vermelho, verde e azul)	(altura, largura, 3)

A rede convolucional exige um canal explícito: de (28, 28) para (28, 28, 1).

Passos 1 e 2 para a CNN

```
x_treino_cnn = x_treino_n.reshape(-1, 28, 28, 1)
x_teste_cnn = x_teste_n.reshape(-1, 28, 28, 1)

print(x_treino_cnn.shape)
```

Saída esperada: (60000, 28, 28, 1)

- Carregar os dados e dividir por 255 é igual ao que já foi feito
- reshape(-1, 28, 28, 1) acrescenta o canal
- O −1 deixa o Python calcular a quantidade de imagens

Passo 3 · Montar a CNN

Rede densa

```
modelo_densa = keras.Sequential([
    keras.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
```

Rede convolucional

```
modelo_cnn = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(10, activation="softmax"),
])
```

Conv2D e MaxPooling2D entram antes do Flatten; o resto da receita é igual.

Parâmetros e summary()

```
modelo_densa.summary()
modelo_cnn.summary()
```

Rede	Parâmetros treináveis
Densa	101.770
Convolucional	34.826

- Parâmetros são os números que a rede aprende
- No resultado de summary(), compare a linha Trainable params
- Depois do treino, Total params pode incluir números do otimizador

A CNN tem menos parâmetros que a rede densa: 34.826 contra 101.770.

Passos 4 e 5 · Compilar e treinar a CNN

```
modelo_cnn.compile(optimizer="adam",
                   loss="sparse_categorical_crossentropy",
                   metrics=["accuracy"])

modelo_cnn.fit(x_treino_cnn, y_treino, epochs=5)
```

- A compilação é igual à da rede densa
- Anote o tempo de treino e compare com o da rede densa

Passo 6 · Avaliar e comparar

```
perda, acuracia = modelo_cnn.evaluate(x_teste_cnn, y_teste, verbose=0)
print(f"Acurácia no teste (CNN): {acuracia:.2%}")
```

Medida	Rede densa	CNN
Acurácia no teste	97,29%	*a preencher*
Tempo de treino	*a preencher*	*a preencher*
Parâmetros treináveis	101.770	34.826

As células "a preencher" dependem de rodar o notebook no Colab: acurácia da CNN e os tempos.

Passo 7 · Os mesmos dígitos nas duas redes

```python
import numpy as np


entrada_densa = (imagem_28 / 255.0).reshape(1, 28, 28)
entrada_cnn = (imagem_28 / 255.0).reshape(1, 28, 28, 1)


p_densa = modelo_densa.predict(entrada_densa, verbose=0)[0]
p_cnn = modelo_cnn.predict(entrada_cnn, verbose=0)[0]


print(f"Rede densa: {np.argmax(p_densa)} ({p_densa.max():.0%})")
print(f"CNN:        {np.argmax(p_cnn)} ({p_cnn.max():.0%})")
```

Compare as respostas das duas redes para o mesmo dígito.

Resumo do Bloco 2: a receita é a mesma

- Os 7 passos são os mesmos nas duas redes
- Só a lista de camadas muda: entram Conv2D e MaxPooling2D
- A entrada da CNN ganha um canal: (28, 28, 1)
- A CNN usa filtros que deslizam pela imagem e aproveitam a vizinhança dos pixels
- Neste exemplo, a CNN tem menos parâmetros: 34.826 contra 101.770

Compare sempre a acurácia, o tempo de treino e o número de parâmetros.

ATIVIDADE OPCIONAL

Repetir a receita com o Fashion-MNIST

Fashion-MNIST: 70 mil imagens de peças de roupa e calçados, 28 × 28, em 10 categorias, no mesmo formato do MNIST.

```
from tensorflow.keras.datasets import fashion_mnist

(x_treino, y_treino), (x_teste, y_teste) = fashion_mnist.load_data()
```

1. Carregar e ver 10 imagens com o nome da categoria (pesquisar a lista de nomes)
2. Treinar a rede densa e a CNN seguindo a mesma receita
3. Comparar acurácia, tempo e parâmetros
4. Descobrir quais categorias as redes mais confundem
5. Escrever 3 ou 4 linhas comparando com o resultado do MNIST

GLOSSÁRIO

Termos da aula

Termo	Significado
Keras	biblioteca para montar redes neurais
TensorFlow	o motor que executa as contas
MNIST	70 mil imagens de dígitos escritos à mão
Pixel	cada ponto da imagem (0 a 255)
Rótulo	a resposta certa de cada imagem
Treino e teste	aprender com um grupo, medir com outro
Rede densa	cada neurônio olha todos os valores anteriores
Flatten	estica a imagem em uma fila de números
Sequential	o modelo como pilha de camadas
Softmax	saídas viram probabilidades (soma 1)

Termo	Significado
compile	define otimizador, erro e métricas
Época	uma passada completa pelos dados de treino
Acurácia	porcentagem de acertos
predict e argmax	calcula as probabilidades e escolhe a maior
Rede convolucional	usa filtros que deslizam pela imagem
Convolução	janela que procura um padrão
Pooling	reduz a imagem mantendo o maior valor do bloco
Canal	camada de cor (1 em cinza, 3 em colorida)
Parâmetros	números que a rede aprende

PRÓXIMOS PASSOS

Da receita do Keras à Unidade 8

A receita se repete em qualquer rede: o que muda é a lista de camadas. A rede convolucional será aprofundada na Unidade 8.

Atividade opcional: repetir a receita com o Fashion-MNIST